Abhinand
Department of Mechanical Engineering
Indian Institute of Technology Madras

IIT Madras

October 20, 2026
Git Revision: 87a4021

All results reproducible from committed artifacts

1. Introduction
2. Related Work
3. Digital Twin Physics
4. Environment Design
5. Reinforcement Learning
6. Results
7. Interactive Workstation
8. Software Engineering
9. Discussion
10. Conclusion

CNC face milling: rotating cutter removes material in multiple passes.

Parameters to optimize:

Spindle speed n [RPM]

Feed per revolution f_r [mm/r]

Axial depth of cut a_p [mm]

Four competing objectives:

Productivity: Maximize $\mathrm{MRR} = v_c \cdot a_p \cdot f_r$

Energy: Minimize total electrical energy/part

Quality: $R_a \leq 1.6$ μm (finished surface)

Tool Life: $V_B \leq 0.3$ mm, avoid breakage

Traditional fixed parameters (handbooks, static NSGA-II) cannot adapt to tool wear, material variation, or geometry changes.

1. **Digital Twin (Physics)**
   - Speed-dependent cutting energy $k_c(v_c) = k_{c0}(v_c/v_{\text{ref}})^m$
   - Extended Taylor tool wear
   - Empirical roughness regression
   - Stochastic breakage
2. **DRL Agent (PPO)**
   - Gymnasium environment
   - Energy-aware reward
   - Conditioned policy (w_2 + material context)
3. **Experimental Rigor**
   - 100-scenario Monte Carlo + 95% CIs
   - Ablation: 7 weights $\times$ 3 seeds
   - 5 classical baselines + receding-horizon
   - Transfer experiment
   - Full reproducibility
4. **Interactive Workstation**
   - 2D stock depletion, micro-topography
   - G-code generation, multi-controller comparison

Table: Capability comparison

Capability	Mo et al. (2025) *JMP 156*	Wang et al. (2025a) *JMS 78*	**This Work**
Dynamic optimization	✓	✓	✓
Tool wear in state	✓	✓	✓
Roughness constraint	✓	✓	✓
Energy as an objective	✗	✗	✓
Speed-dependent k_c	✗	✗	✓
Domain randomization	✗	✗	✓
Conditioned policy	✗	✗	✓
Transfer experiment	✗	✗	✓
Receding-horizon baseline	✗	✗	✓
Monte Carlo (100 scenarios)	✗	✗	✓
Interactive workstation	✗	✗	✓

Key differentiator: Speed-dependent k_c creates genuine energy/throughput trade-off; conditioned policy covers frontier in 1 run.

$$v_c = \frac{\pi D n}{1000} \tag{1}$$

$$\mathrm{MRR} = v_c \cdot a_p \cdot f_r \tag{2}$$

$$t_{\text{cut}} = \frac{L_{\text{cut}}}{n f_r} \tag{3}$$

$$\Delta V_B = C_{\text{wear}} v_c^{\alpha} f_r^{\beta} a_p^{\gamma} t_{\text{cut}} \tag{4}$$

$$R_a = k_1 f_r^2 + k_2 V_B + \frac{k_3}{v_c} \tag{5}$$

$$k_c(v_c) = k_{c0} \left(\frac{v_c}{v_{\text{ref}}} \right)^m \tag{6}$$

$$P_{\text{total}} = P_{\text{idle}} + k_c(v_c) \cdot \mathrm{MRR} \tag{7}$$

$$E_{\text{pass}} = P_{\text{total}} \cdot t_{\text{cut}} \tag{8}$$

Key Insight: Interior Minimum

With $m = 0.6 > 0$:

- Idle energy $\propto 1/(n f_r) \downarrow$ with speed
- Cutting energy $\propto n^m \uparrow$ with speed
- Part energy $E_{\text{part}}(v_c)$ has **interior minimum**
- Energy weight w_2 selects operating point

Validated by a unit test on the part energy minimum

Two Registrations

`MillingDTEnv-v1`: base, 8-D obs

`MillingDTEnv-w2-v1`: conditioned, 11-D obs (w_2, material context)

Observation (8 features, each onto $[0, 1]$)

tanh-squashed V_B, κ_{rem}

previous n, f_r, a_p (to the machine envelope)

previous $\text{MRR}/\text{MRR}_{\text{max}}$, tanh-squashed R_a

tool-broken flag

conditioned adds w_2, C_{wear}, k_1

Action: $[-1, 1]^3 \rightarrow$ clipped $\rightarrow$ machine envelope

Reward per step

$$r = 5 \left(w_1 \frac{\text{MRR}}{\text{MRR}_{\text{max}}} - w_2 \frac{E_{\text{pass}}}{E_{\text{ref}}} \right) + 2\,\Delta\left(\tfrac{\kappa_{\text{removed}}}{\kappa_{\text{total}}}\right) \tag{9}$$

$$- 50\,\mathbb{1}[V_B > V_{B,\text{safe}}] - 100\,\mathbb{1}[\text{broken}] \tag{10}$$

$$- 50\,\mathbb{1}[\text{finish} \wedge R_a > R_{a,\text{safe}}] - 0.05 \tag{11}$$

plus $+200$ for a part finished in spec, and a Pareto margin bonus of at most $+5$.

The objective term carries a scale of 5 so that it competes with the shaping terms. Without it the shaping decides the policy, w_2 has no effect, and the ablation produces 21 identical policies.

E_{ref} is recomputed for every scenario from that scenario's diameter, cutting length and energy coefficients.

Workpiece: $\kappa_{\text{total}} \in [2, 15]$, $D \in [40, 150]$, $L_{\text{cut}} \in [50, 200]$

Material/Tool: C_{wear} at $[0.6, 1.4]\times$ nominal, k_1, k_2, k_3 at $[0.8, 1.2]\times$ nominal

Initial wear: $V_{B,0} \sim \mathcal{U}[0, 0.8]V_{B,\max}$

Shared tables: Same `config/scenarios.py` for training + Monte Carlo eval

Measured: the observations the policy sees on the training and the evaluation sets agree to 0.041 in the mean (std = 1) — the same distribution

Table: Hyperparameters

Parameter	Value
Policy	MlpPolicy (2×256)
Learning rate	3×10^{-4} (linear)
n_steps	2048
Batch size	64
n_epochs	10
γ / GAE λ	0.99 / 0.95
Clip range	0.2
Entropy coeff	0.02
VecNormalize	norm_obs=True, norm_reward=True
Envs / Device	4 (`SubprocVecEnv`) / CPU

Training Runs

Primary PPO: 1M steps, seed 42, randomize=True → `models/PPO_seed42/`

Conditioned PPO: 1M steps, seed 42, on `MillingDTEnv-w2-v1`

Ablation Grid: 7 w_2 × 3 seeds = 21 policies, 150k steps each

Heartbeat callback every 1000 steps → stall detection in `run_experiments.py`

Table: Monte Carlo results

Metric	Mean	Std	95% CI
MRR (mm^3/min)	1239	621	[1116, 1363]
Energy (kJ)	68.2	48.7	[58.5, 77.9]
Cycle time (s)	19.4	14.1	[16.6, 22.2]
Max R_a (μm)	1.475	0.235	[1.428, 1.521]
Max V_B (mm)	0.124	0.068	[0.111, 0.138]
Roughness viol. (worst)	39.0%		[29%, 49%]
Roughness viol. (finished)	0.0%		[0%, 0%]
$V_B > 0.3$ mm	0.0%		[0%, 0%]
Tool breakage	1.0%		[0%, 3%]
Parts completed	99.0%		

Key Findings

- Finished surface always in spec (0% violations)
- Wear never exceeded (0% violations)
- 99% of parts completed
- 61% of passes exceed the safety margin $\rightarrow$ the policy pushes and recovers by the finish pass
- Mean op. point: 7241 RPM, 0.254 mm/r, 2.11 mm, 4 passes

Table: Ablation: mean ± std over 3 seeds, 50 scenarios each

w_2	MRR	Energy (kJ)	Cycle (s)	n (RPM)	f_r	Frontier
0.0	999.9±31	82.8±0.7	45.8±2	6974	0.177	dominated
0.2	990.5±137	76.3±2.2	31.2±6	7307	0.186	dominated
0.4	1069.0±40	71.5±0.4	22.9±1	7205	0.212	✓
0.5	1026.2±57	71.0±0.7	23.2±0.5	7115	0.208	✓
0.6	1008.4±23	68.3±0.8	21.6±1	6775	0.223	✓
0.8	888.3±58	65.8±1.4	23.1±1	6307	0.222	✓
1.0	876.6±43	65.4±1.7	22.7±1	6321	0.220	✓

Frontier Analysis

Across frontier ($w_2 \in [0.4, 1.0]$): MRR falls 18.0% (1069 → 877), Energy falls 8.5% (71.5 → 65.4). Asymmetric trade-off!

Dominated points ($w_2 = 0.0, 0.2$) have **higher** energy than $w_2 = 0.4$ – pushing speed too high increases $k_c(v_c)$ more than idle energy saved.

Table: Conditioned vs Ablation MRR

w_2	Conditioned, 1M MRR	Per-weight, 150k MRR	Energy gap
0.0	1213.6	999.9	−15.9%
0.2	1215.3	990.5	−9.5%
0.4	1205.1	1069.0	−4.4%
0.5	1191.0	1026.2	−4.2%
0.6	1169.3	1008.4	−1.0%
0.8	1130.8	888.3	+2.0%
1.0	1096.0	876.6	+2.6%

Result

One run of 1,000,000 steps beats the 21 policies of 150,000 steps each (3,150,000 steps total): 13% to 27% more removal rate for at most 3% more energy.

The weight stays a runtime query, not a retrain.

Observation carries: $w_2 \in [0, 1]$ + material context (C_{wear}, k_1), drawn per episode at `reset()`.

Table: Baseline comparison

Policy	MRR	Energy (kJ)	Cycle (s)
PPO (primary)	877.7	18.5	5.7
Receding-Horizon	**981.7**	**18.0**	**4.8**
NSGA-II Static	732.9	18.8	6.4
Rule-Based	719.8	18.7	6.6
Constant-Safe	1.6	1506.8	3000.0
Constant-High	1099.6	17.8	4.3 (violates)

Same 50 episodes of the nominal condition for all six policies

The four static policies end outside the roughness safety limit (reward -46); only the controller stays inside ($+203.5$)

PPO $+204.1$ **vs controller** $+203.5$: level on the reward, and the controller is ahead on MRR, energy and cycle time

No DRL advantage yet: next step is to initialize PPO from the controller $\rightarrow$ fine-tune

Target Material (Unseen)

Harder alloy: $k_1 \times 1.25$, $k_2 \times 1.30$

Faster-wearing tool: $C_{\text{wear}} \times 2.0$

Source policy NEVER saw this material

Table: Transfer results

Arm	Reward	Ra violations	Parts done
Source zero-shot	183.0	100.0%	93.3%
Source fine-tuned (50k)	182.9	100.0%	93.3%
Target from scratch (50k)	196.7	0.0%	100.0%

Fine tuning is the wrong tool

Fine tuning moves the reward from 183.0 to 182.9 and removes no violation. Training on the harder material from scratch removes all of them.

2D Stock Depletion: Cylindrical billet cross-section; pass-by-pass cutout; remaining κ, a_p; tool engagement

Micro-Topography: Feed-mark cusp $y_{\text{cusp}}(x) \approx x^2/(8r_\epsilon) + \text{noise}$; ISO 1302 R_a overlay; 1.6 μm ceiling

G-Code Generator: ISO 6983 Fanuc/Siemens compliant; exact PPO sequence; tool/diameter aware

Multi-Controller: Side-by-side PPO vs NSGA-II vs Rule-Based vs Constant-High; live telemetry

Cached Loading: `@st.cache_resource` loads model once per session

```
streamlit run app.py
```

Module Structure

- `config/`: Physics, scenarios, paths (single source)
- `env/`: Gymnasium env, 2 registrations
- `agent/`: Train, eval, ablation, transfer, pipeline
- `baselines/`: 5 classical + receding-horizon
- `sim/`: Stock depletion, topography, G-code
- `viz/`: Shared plotting utilities

Reproducibility

- Global seeding (Python, NumPy, PyTorch, CUDA)
- Shared scenario tables (train = eval)
- `summarize_results.py` regenerates `RESULTS.md`

Test Coverage (26 tests)

- 19 env tests: clipping, truncation, soft norm, constants, overrides, energy norm, Gym checker, randomization, scenario scales, speed dependence, energy minimum, shared physics
- 2 reward tests: golden value (1e-9), finishing penalty
- 3 integration: train→eval roundtrip, conditioned obs, baseline artifact
- 3 callback: heartbeat, failure report, interval

Code Quality

- No unused imports (AST verified)
- Literature citations in `physics_constants.py`

What Worked

- Speed-dependent k_c creates genuine trade-off
- Domain randomization $\rightarrow$ 99% completion, 0% finished violations
- Conditioned policy: 1 run replaces 21 (beats them at 1/3 the steps)
- Receding-horizon = strong, fair baseline
- Tests catch physics & reward regressions

Limitations

- PPO is level with receding-horizon, not ahead of it
- Ablation: 150k steps/weight (vs 1M primary)
- Reward charges a pass, metric is a part
- Sim-only, no real calibration yet

Future Work

- Controller-initialized RL (BC + fine-tune)
- Sim-to-real: calibrate, add noise/delay
- True multi-objective RL (Pareto, hypervolume)
- Online w_2 adaptation for energy budget

Digital Twin with speed-dependent cutting energy, tool wear, roughness, stochastic breakage

Gymnasium Environment with domain randomization, conditioned observations, energy-aware reward

PPO Policies with rigorous evaluation: Monte Carlo (100 scenarios, 95% CIs), ablation frontier, conditioned policy

Strong Baselines including receding-horizon controller (re-optimizes every pass)

Transfer Experiment showing that fine tuning does *not* adapt, but training across materials does

Interactive Workstation: 2D stock depletion, micro-topography, G-code, multi-controller comparison

Full Reproducibility: every result table from committed artifacts by one script

Key Scientific Finding

The speed-dependent specific cutting energy creates a **genuine energy/throughput trade-off**, and DRL learns to navigate it pass-by-pass while respecting surface quality and tool life constraints. Across the measured frontier the removal rate falls 18% while the energy of the part falls 8.5%. The conditioned policy makes the whole trade-off accessible at runtime with a single model.

Task	Command
Tests	`pytest tests/ -v`
Train PPO	`python agent/train_rl.py -steps 1000000`
Train conditioned	`python agent/train_rl_conditioned.py -steps 1000000`
Monte Carlo eval	`python agent/evaluate.py`
Ablation grid	`python agent/train_rl_ablation.py -steps 150000`
Baselines	`python baselines/run_baselines.py`
Transfer exp.	`python agent/transfer_experiment.py`
Regenerate results	`python agent/summarize_results.py`
Launch workstation	`streamlit run app.py`
Full pipeline	`python agent/run_pipeline.py -steps 1000000`

Thank You

Questions & Discussion

Repository: `G:/My Drive/IITM/AI in Mfg`
ME6324 – AI in Manufacturing | IIT Madras